

DIETI UNIVERSITA' DEGLI STUDI DI NAPOLI FEDERICO II
DIPARTIMENTO DI INGEGNERIA ELETTRICA E DELLE TECNOLOGIE DELL'INFORMAZIONE

BASI DI DATI I

A.A. 2022/2023

Prof. Silvio Barra

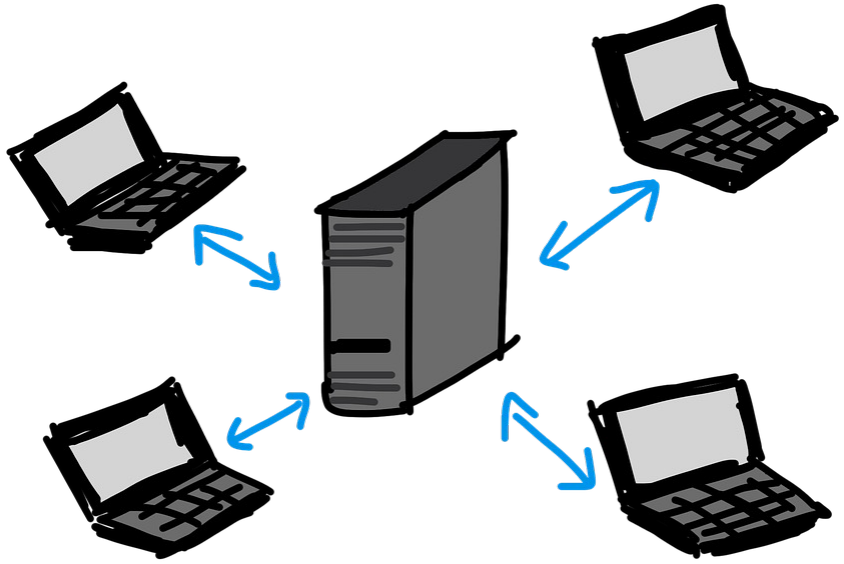
DBMS: DAL MONOLITICO AL DISTRIBUITO

- La maggior parte dei sistemi informativi sta sempre più adottando soluzioni distribuite, contrapponendosi al sistema monolitico che prevedeva un sistema HW e SW estremamente integrate l'uno con l'altra
- L'evoluzione dei DBMS ha avuto una analoga evoluzione
  - Dal monolitico al modulare/distribuito con architetture di tipo client-server
  - Gli attuali ambienti di cloud computing consistono di migliaia di server dislocati geograficamente che gestiscono enormi moli di dati (Big Data) per conto degli utenti



CLIENT-SERVER

- Nelle architetture client-server si hanno
  - Un modulo **client** progettato per essere eseguito su un pc dell'utente
    - Dovendo gestire l'interazione con l'utente, è spesso fornito di GUI user-friendly
  - Un modulo **server** che si occupa di permettere l'accesso agli utenti ad una serie di servizi, quali la memorizzazione, l'elaborazione e la generazione di informazioni



MODELLI DEI DATI



MODELLI ED ASTRAZIONE DATI

- L'approccio database ha la caratteristica fondamentale di fornire una forte astrazione dei dati, nascondendo tutti i dettagli di memorizzazione non necessari agli utenti.

- L'astrazione dei dati si ottiene per mezzo di un **data model** (modello dei dati) che consiste di:
  - Una **struttura** della base di dati
    - ***Es:*** i **tipi dei dati**, i **vincoli** che specificano restrizioni sui dati per renderli validi e le **relazioni** che intercorrono tra i dati
  - Le **operazioni** per manipolare le strutture
    - ***Es:*** Inserzione, cancellazione, modifica, ritrovamento di un oggetto.
  - Un **comportamento dinamico (o aspetto)** per permettere all'utente di definire operazioni sui dati
    - Un'operazione CALCOLA_MEDIA_VOTO che calcola la media dei voti e che può essere applicata a un oggetto studente.



COSA DEVE FARE UN MODELLO

Un modello deve:

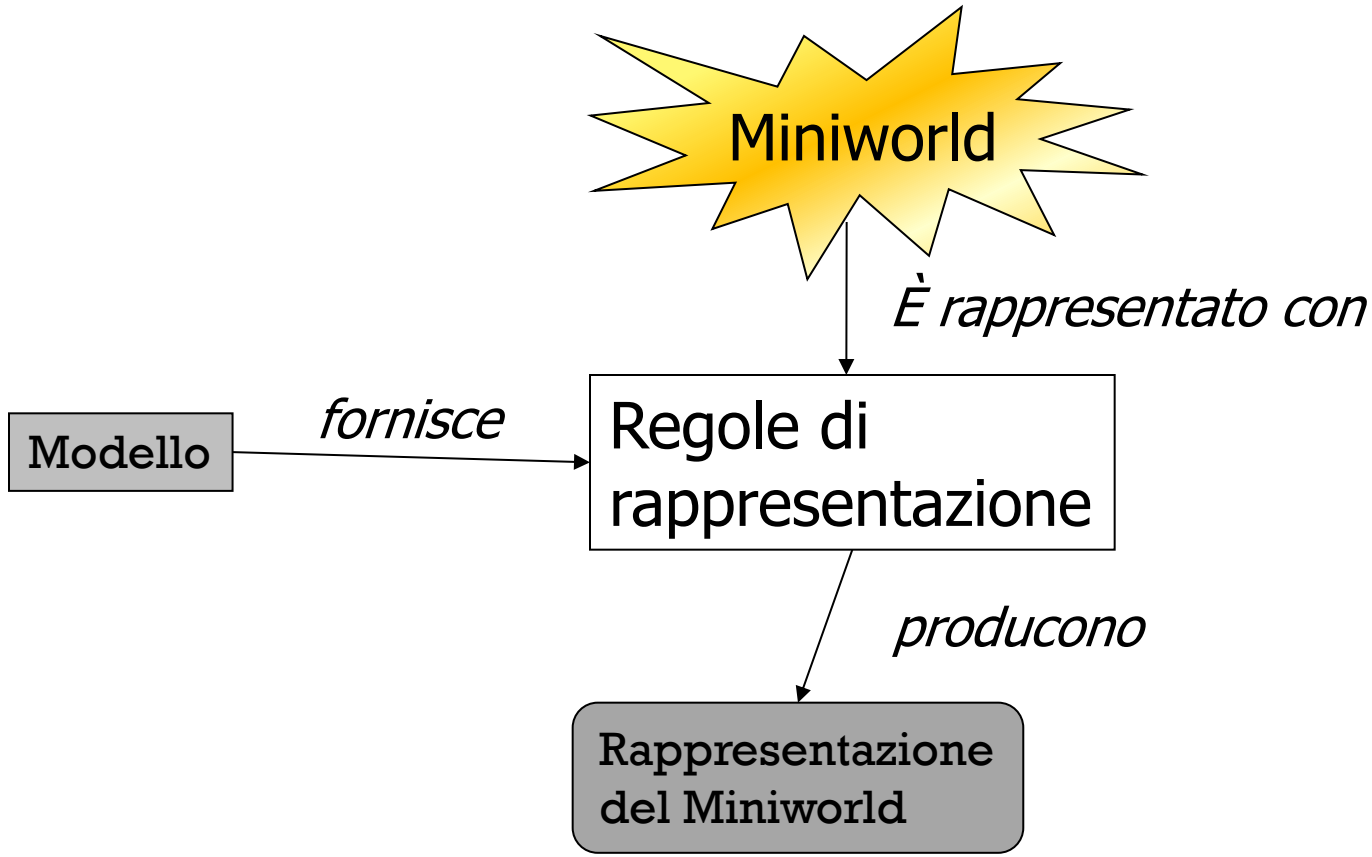
- Rappresentare una certa realtà:
  - ***Es:*** Una mappa rappresenta una porzione di territorio.

    È quindi un modello del territorio, che rappresenta alcune caratteristiche, nascondendone altre.

- Fornire un insieme di strutture simboliche per descrivere la rappresentazione della realtà:
  - ***Es:*** Una mappa ha una serie di simbolismi grafici per rappresentare delle entità reali.

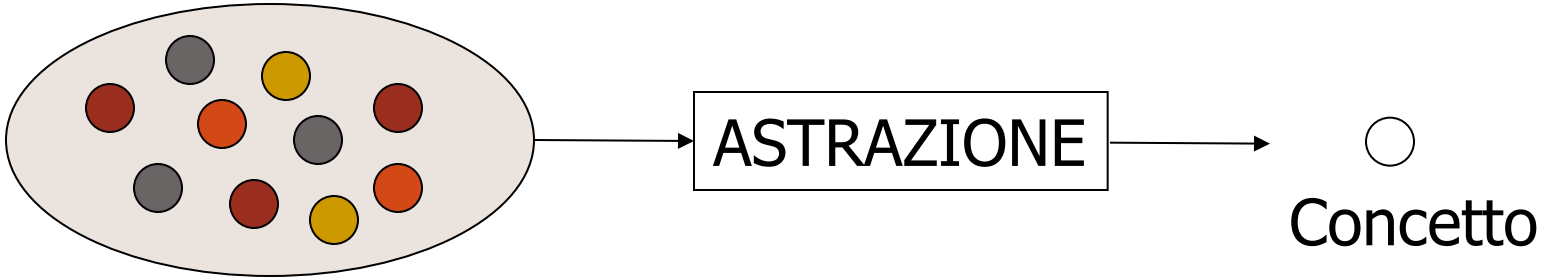


COS'È UN MODELLO (SCHEMA)



ASTRAZIONE

- L'astrazione è un procedimento mentale che sostituisce con un concetto un insieme di oggetti in base ad alcune loro proprietà:



Esempio:
l'astrazione consente di definire il concetto di "automobile". Grazie ad esso è possibile descrivere e riconoscere tutte le automobili della realtà.

ASTRAZIONE E MODELLI

- L'astrazione è importante perché permette di comunicare ed elaborare una rappresentazione del miniworld.
- Per comunicare ed elaborare tale rappresentazione si utilizzano dei modelli concettuali e logici.
- Un modello concettuale fornisce i simbolismi per rappresentare concetti astratti in modo indipendente da qualsiasi elaboratore.
- Un modello logico traduce le strutture concettuali in strutture logiche processabili da un DBMS.



CATEGORIE DI DATA MODEL

È possibile categorizzare i vari data model proposti secondo i concetti utilizzati per descrivere la struttura del database.

- I data model di **alto livello** o **concettuali** forniscono concetti che sono vicini al modo di percepire i dati degli utenti.
  - Anche noti come **entity-based** o **object-based** data models
- I data model di **basso livello** o **fisici** forniscono concetti che descrivono dettagli su come i dati sono memorizzati.
- I data model **rappresentazionali** o **implementazione** forniscono concetti comprensibili agli utenti finali, ma che non sono troppo lontani dal modo in cui i dati sono fisicamente organizzati.
- I data model **auto-descrittivi** combinano la descrizione dei dati con i valori dei dati stessi
  - Ad esempio, come si fa con l'XML



DATA MODEL CONCETTUALI (ALTO LIVELLO)

- I data model di alto livello usano concetti quali **entità**, **attributi** e **relazioni**:
  - Un'entità rappresenta un oggetto o concetto del mondo reale.
    - ***Es:*** Un impiegato o un progetto.
  - Un attributo rappresenta qualche proprietà importante che descrive ulteriormente un'entità.
    - ***Es:*** Il nome o lo stipendio di un impiegato.
  - Una relazione tra due o più entità rappresenta un'interazione tra le entità.
    - ***Es:*** Una relazione "lavora su" tra un impiegato e un progetto.
- Il più comune data model di alto livello è il modello Entità-Relazione (Entity-Relationship).



DATA MODEL RAPPRESENTAZIONALI

- I **data model rappresentazionali**, per la facilità con cui possono essere implementati, comprendono i tre data model più usati dai DBMS commerciali:
  - Il **modello relazionale** (i dati sono incapsulati in record a struttura fissa. La relazione è rappresentata da una tabella.)
  - Il **modello reticolare** (si basa sull'utilizzo di grafi e si basano sui concetti di record e set. Utilizzato per limitare la ridondanza.) - *legacy*
  - Il **modello gerarchico** (utilizza strutture ad albero con i record come nodi e le associazioni come archi. Ha problemi per quanto riguarda la ridondanza dei dati.) - *legacy*
- Tali modelli rappresentano i dati usando strutture di record. Per tale motivo sono chiamati anche **modelli dei dati record-based**.
- I **modelli dei dati object-oriented**, grazie alla forte astrazione dei dati, costituiscono una nuova famiglia di data model, posti a metà strada tra il rappresentazionale e il concettuale.



ESEMPI DI DATA MODEL RAPPRESENTAZIONALI



gerarchico

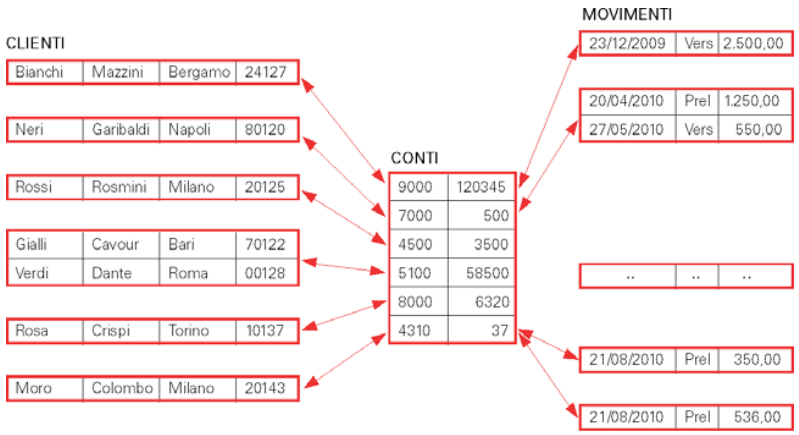
Activity Code	Activity Name
23	Patching
24	Overlay
25	Crack Sealing

Key = 24

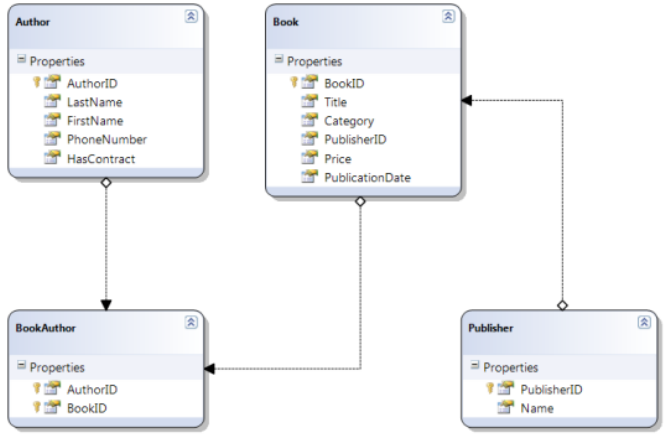
Activity Code	Date	Route No.
24	01/12/01	I-95
24	02/08/01	I-66

Date	Activity Code	Route No.
01/12/01	24	I-95
01/15/01	23	I-495
02/08/01	24	I-66

relazionale



reticolare



object-based

DATA MODEL FISICI

- I **data model fisici** descrivono come sono memorizzati i dati nel calcolatore rappresentando informazioni quali formati di record, ordinamenti di record, percorsi di accesso.
  - Un *percorso di accesso* è una struttura che rende efficiente la ricerca di particolari record di database.
  - L'***indice*** è un esempio di percorso di accesso che garantisce l'accesso diretto ai dati tramite un termine indice o una parola chiave
    - Gli indici possono essere poi organizzati in maniera lineare, gerarchica o in altra maniera



SCHEMI, ISTANZE E STATO DI UNA BASE DI DATI



SCHEMI E ISTANZE DI DATABASE

- In qualsiasi data model è necessario distinguere tra la descrizione del database ed il database stesso.
  - La differenza è simile a quella tra tipi e variabili nei linguaggi di programmazione.

- La descrizione è detta **schema** della base di dati e corrisponde alla struttura logica del database.
  - Talvolta chiamata **intensione della base di dati**

- Un'**istanza** è il contenuto del database in un particolare istante di tempo.
  - Talvolta chiamata **estensione della base di dati**



SCHEMI DI BASI DI DATI

- **La descrizione del database è detta schema (o metadati).**
- Lo schema del database è specificato in fase di progetto e non ci si aspetta che cambi frequentemente.
  - Per rappresentare uno schema si crea un diagramma di schema, secondo opportune convenzioni definite dal data model.
  - Un **diagramma di schema** visualizza la struttura dei record ma non le reali istanze dei record.

- Il DBMS memorizza lo schema nel catalogo per poterne fare riferimento ogni qualvolta gli occorre.



SCHEMI DL DATABASE: *ESEMPIO DB UNIVERSITÀ*

STUDENTE			
Nome	**Matricola**	**Anno_Corso**	**Corso_Laurea**

INSEGNAMENTO			
Nome_Ins	**Cod_Ins**	**CFU**	**Dipartimento**

PREREQUISITI	
Cod_Ins	**Cod_Prop**

VOTAZIONE		
Matricola	**Cod_Ins**	**Voto**

- Ogni oggetto nello schema è detto **costrutto di schema**.
  - ***Es:*** STUDENTE o CORSO sono ciascuno un costrutto di schema.



ISTANZE DI DATABASE

- **I dati reali in un DB in uno specifico istante di tempo costituiscono l'istanza della base di dati**
  - Al contrario dello schema questi possono cambiare frequentemente.
  - Es: Il DB UNIVERSITA' cambia ogni volta che si inserisce un nuovo studente o si registra un nuovo esame per uno studente.
- In un certo stato della base di dati, ogni costrutto ha il proprio insieme corrente di istanze
  - Es: Il costrutto STUDENTE conterrà l'insieme delle singole entità (*record*) di ogni studente
- Per un dato schema di database possono essere costruiti diversi stati di database.
  - Ogni volta che si inserisce o cancella un record o si modifica il valore di un data item, si cambia lo stato del database

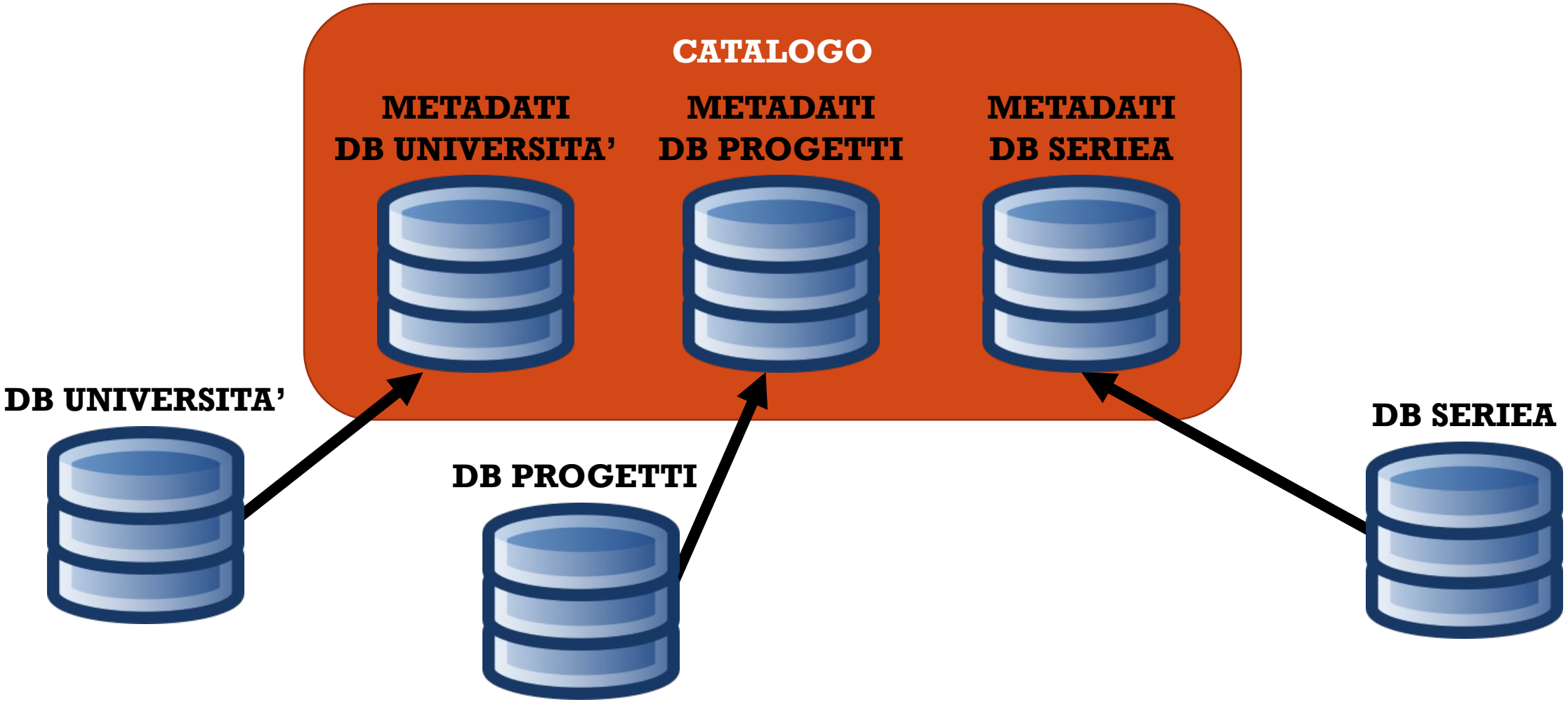


L' IMPORTANZA DELLO SCHEMA DI DATABASE

- La distinzione tra **schema** e **istanza** è di notevole importanza
  - Quando si crea un nuovo database, si specifica al DBMS lo schema del database: in tale fase il db è nello *"**stato vuoto**"*, senza dati.
  - Si passa nello *"**stato iniziale**"*, quando si inseriscono i dati per la prima volta (popolazione della base di dati).
  - Da questo momento in poi ogni inserimento, cancellamento o modifica porterà il DB in un nuovo stato.
- Il DBMS è parzialmente responsabile nel garantire che ogni stato del DB sia **valido**, cioè che soddisfi la struttura ed i vincoli specificati nello schema.
- Quindi specificare uno schema corretto è estremamente importante
  - Lo schema deve essere progettato con la massima cura.



PostgreSQL


CATALOGO
METADATI
DB UNIVERSITA'
METADATI
DB PROGETTI
METADATI
DB SERIEA
DB UNIVERSITA'
DB PROGETTI
DB SERIEA

EVOLUZIONI

- Lo schema della base di dati non dovrebbe cambiare frequentemente, ma ad ogni modo può accadere che alcune modifiche siano necessarie, soprattutto quando:
  - Cambiano i requisiti dell'applicazione.
  - Cambia il mini-world che stiamo rappresentando.
- In tali casi, lo schema deve cambiare.
  - **Evoluzione dello schema della base di dati.**

- La maggior parte dei moderni DBMS include operazioni per permettere l'evoluzione dello schema **MENTRE** la base di dati è operativa.



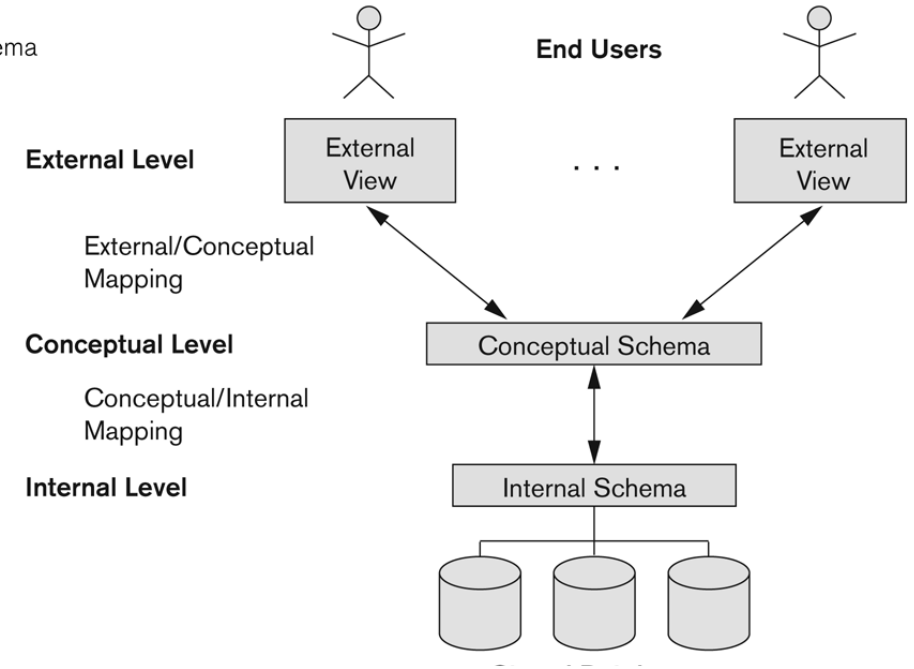
ARCHITETTURA DEI R-DBMS



L'ARCHITETTURA A TRE LIVELLI

(ARCHITETTURA THREE-SCHEMA)

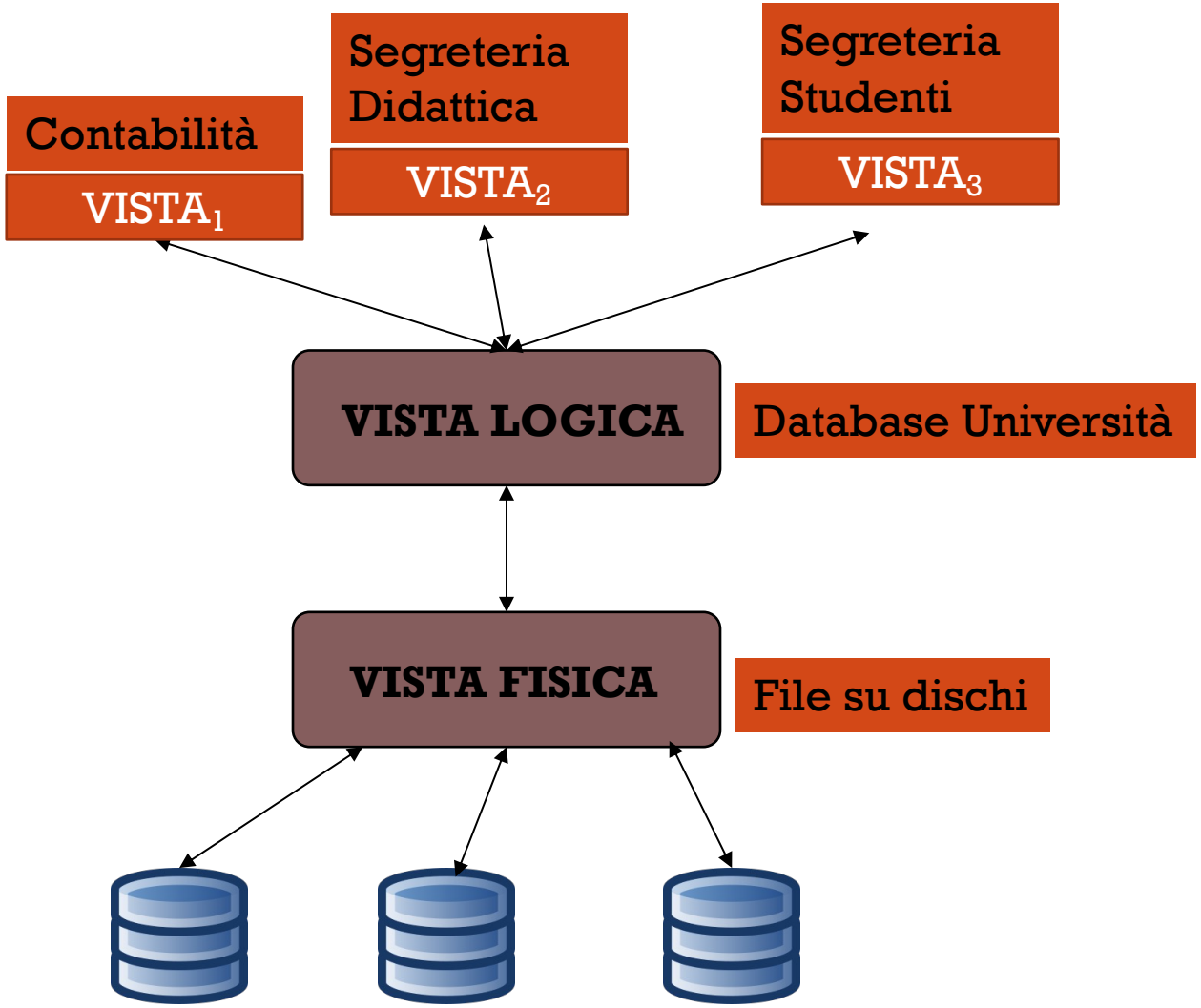
Figure 2.2
The three-schema architecture.



- Ricordiamo le caratteristiche principali dell'approccio di database:
  - Supporto di viste multiple per gli utenti;
  - Uso di un catalogo per memorizzare la descrizione del DB (lo schema);
  - Isolamento tra programmi e dati.

- L'architettura per database system chiamata **three-schema** aiuta ad ottenere tali caratteristiche, definendo il database in tre livelli e separando quindi le applicazioni utente dal database fisico.

L'ARCHITETTURA THREE-SCHEMA: UN'*ESEMPIO*



Gli schemi possono essere specificati a tre livelli:

- Interno (**Internal Schema)**
  - Per descrivere le strutture di storing fisiche (*physical data model*)
- Concettuale (**Conceptual Schema**)
  - Descrive la struttura e i vincoli del database
- Esterno (**External Schema)**
  - Descrive le varie viste per gli utenti

LIVELLO INTERNO

Livello interno come schema interno:

- Descrive la struttura di memorizzazione fisica del DB.

- Lo schema interno usa un data model fisico e descrive i dettagli completi del data storage e gli access paths del DB.

- ***Esempio:***
  Dividi i record del file studenti in tre partizioni sui dischi 5, 6 e 7.



LIVELLO CONCETTUALE

Livello concettuale come schema concettuale:

- Descrive la struttura del DB per una comunità di utenti.
- Lo schema concettuale nasconde i dettagli delle strutture di storage fisico e si concentra sulla descrizione delle entità, tipi di dati, relazioni, operazioni utente e vincoli.
- Può usare un data model ad alto livello o uno di implementazione.

- ***Esempio:***
  type Studente= **record**
  nome : **string**;
  matricola : **string**;
  anno : **string**;
  end;



LIVELLO ESTERNO

Livello esterno come schema esterno:

- Definisce un sottoinsieme del DB per una particolare applicazione.
- Include più schemi esterni o viste utente.
- Usa un data model di alto livello o di implementazione.
- Ogni schema esterno descrive la parte del DB cui è interessato un particolare utente e nasconde il resto del DB a quel gruppo.

- ***Esempio:***
  La segreteria didattica ha necessità di vedere tutte le informazioni sugli esami, ma non deve vedere informazioni sugli stipendi.



I MAPPING TRA I LIVELLI DELL'ARCHITETTURA

- I tre schemi sono solo delle descrizioni di dati: gli unici dati che realmente esistono sono a livello fisico.

- In un DBMS basato sull'architettura three-schema, ogni gruppo di utenti utilizza una propria vista esterna.

- Un mapping è un processo di trasformazione delle richieste e dei risultati. Il DBMS trasforma:
  - una richiesta specificata su uno schema esterno…
  - …in una richiesta secondo schema concettuale e poi…
  - …in una richiesta sullo schema interno per procedere sul database memorizzato.



INDIPENDENZA LOGICA E FISICA DEI DATI



INDIPENDENZA DEI DATI

L'architettura three-schema è utile per evidenziare il concetto di indipendenza dei dati, ovvero la capacità di cambiare lo schema a un livello del database senza dover cambiare lo schema al livello superiore.

Si definiscono due tipi di indipendenza dei dati:

- ***Indipendenza logica dei dati***:
  - lo schema concettuale può essere cambiato senza dover cambiare gli schemi esterni o i programmi applicativi.
- ***Indipendenza fisica dei dati***:
  - lo schema interno può essere cambiato senza dover cambiare gli schemi concettuali (o esterni).



INDIPENDENZA LOGICA

- Indica la capacità di cambiare lo schema concettuale senza dover cambiare lo schema esterno e gli applicativi correlati

- Lo schema concettuale può essere cambiato per espandere oppure per ridurre il database (aggiungendo, rimuovendo o modificando un tipo di record, un data item o un vincolo).

- Se si elimina un tipo di record, gli schemi esterni che si riferiscono solo ai dati restanti non devono essere alterati.
  - Se uno schema ad un livello più basso viene modificato, soltanto il **mapping tra questo schema e quelli di livello più alto necessitano di essere cambiati**;
  - i programmi applicativi che fanno riferimento agli schemi esterni sono indifferenti alle modifiche, siccome si riferiscono solo agli schemi esterni.



SCHEMI DL DATABASE: *ESEMPIO DB UNIVERSITÀ*

STUDENTE			
Nome	**Matricola**	**Anno_Corso**	**Corso_Laurea**

INSEGNAMENTO			
Nome_Ins	**Cod_Ins**	**CFU**	**Dipartimento**

PREREQUISITI	
Cod_Ins	**Cod_Prop**

VOTAZIONE		
Matricola	**Cod_Ins**	**Voto**

- Ogni oggetto nello schema è detto **costrutto di schema**.
  - ***Es:*** STUDENTE o CORSO sono ciascuno un costrutto di schema.



INDIPENDENZA LOGICA: *ESEMPIO*

Lo schema esterno

ESAMI SUPERATI	Nome	Matricola	ESAMI	
			DENOMIN.	VOTO
	Rossi	N86000484	Basi di Dati	24
			Object Orientation	28
	Verdi	N86000323	Algoritmi	30
			Sistemi Biometrici	27

non dovrebbe essere alterato cambiando il file votazione da

VOTAZIONE

NOME	DENOMIN.	VOTO
Rossi	Basi di Dati	24
Rossi	Object Orientation	28
Verdi	Algoritmi	30
…	…	…

a

VOTAZIONE

NOME	MATRICOLA	DENOMIN.	VOTO
Rossi	N86000484	Basi di Dati	24
Rossi	N86000484	Object Orientation	28
Verdi	N86000323	Algoritmi	30
…		…	…

INDIPENDENZA FISICA

- Indica la capacità di cambiare lo schema interno senza dover cambiare lo schema concettuale

- Un cambiamento dello schema interno può essere dovuto alla riorganizzazione di qualche file fisico (***es:*** creando ulteriori strutture di accesso, o modificando degli indici), per migliorare l'esecuzione del ritrovamento o dell'aggiornamento.
  - Se i dati non subiscono alterazioni, non è necessario cambiare lo schema concettuale.

- L'indipendenza fisica si ottiene più facilmente di quella logica.



INDIPENDENZA FISICA: *ESEMPIO*

- Fornire un percorso di accesso per migliorare il ritrovamento dei record del file **CORSO** con **Denominaz.** e **Semestre** non dovrebbe richiedere variazioni a una query come

"ritrova tutti i corsi tenuti nel secondo semestre"

sebbene la query possa essere eseguita in maniera più efficiente.



REALIZZAZIONE INDIPENDENZA LOGICA E FISICA

- L'**indipendenza fisica** dei dati si realizza quasi sempre, nella maggior parte delle basi di dati e nella maggior parte degli ambienti
  - I dettagli di basso livello sono effettivamente nascosti agli utenti.
  - Le applicazioni ignorano dettagli di memorizzazione, posizionamento, compressione, …
- L'**indipendenza logica** è più complicata da realizzare
  - Apportare modifiche alla struttura dei dati e ai vincoli, senza incidere sulle applicazioni è alquanto impegnativo.

- Con un'architettura a più livelli, è necessario aggiungere ulteriori informazioni nel catalogo del DBMS su come mappare le richieste tra i diversi livelli.
- L'indipendenza dei dati avviene perché se uno schema ad un determinato livello $\mathbf{L_i}$ muta, non muta lo schema al livello $\mathbf{L_{i+1}}$ (o $\mathbf{L_{i-1}}$), bensì muta il mapping tra i livelli.



L'ARCHITETTURA THREE-SCHEMA: VANTAGGI E SVANTAGGI

VANTAGGI

- L'architettura three-schema consente facilmente di ottenere una reale indipendenza dei dati.
- Il database risulta più flessibile e scalabile.

SVANTAGGI

- Il catalogo del DBMS multi-livello deve avere dimensioni maggiori, per includere informazioni su come trasformare le richieste e di dati tra i vari livelli.
- I due livelli di mapping creano un overhead durante la compilazione o esecuzione di una query di un programma, causando inefficienze nel DBMS.



DBMS LANGUAGES



LINGUAGGI DBMS

- Abbiamo visto in precedenza, quanti e quali sono gli utenti tipici di un DBMS
  - Amministratori, progettisti, utenti finali, operatori, sviluppatori, …
- *Un DBMS deve fornire ad ogni categoria di utenti delle interfacce e dei linguaggi adeguati.*
- Alla fase di progetto del DB segue quella di specifica degli schemi concettuale ed interno e di qualsiasi mapping tra i due.



DATA DEFINITION LANGUAGE (DDL)

- Nei DBMS dove NON C'È una netta separazione tra livelli, progettisti e DBA usano un **data definition language (DDL)** per definire lo schema concettuale e lo schema interno.
- Il compilatore del DDL (contenuto nel DBMS) ha la funzione di:
  - Elaborare le istruzioni DDL
  - Identificare le descrizioni dei costrutti dello schema.
  - Memorizzare la descrizione dello schema nel catalogo del DBMS.



STORAGE DEFINITION LANGUAGE (SDL)

- Dove c'è una chiara distinzione tra livello concettuale ed interno si usa:
  - uno **storage definition language** (**SDL**) per specificare lo schema interno,
  - il **DDL** per specificare soltanto lo schema concettuale.
- I mapping tra i due schemi possono essere specificati in uno qualsiasi dei due linguaggi.
- Nella maggior parte dei DBMS non c'è uno specifico linguaggio con il ruolo di SDL, ma i DBA procedono definendo una serie di parametri e specifiche relative alla memorizzazion



VIEW DEFINITION LANGUAGE (VDL)

- Nelle architetture three-schema viene usato un **view definition language** (**VDL**) per specificare le viste utente e i loro mapping sullo schema concettuale.
- Nella maggior parte dei DBMS, il **DDL** è utilizzato sia per la definizione degli schemi esterni che per gli schemi concettuali
- Negli R-DBMS, si utilizza l'SQL come linguaggio per la definizione delle viste, come output di una determinata interrogazione



DATA MANIPULATION LANGUAGE (DML)

- Una volta che gli schemi sono compilati ed il DB è riempito di dati, il DBMS fornisce un **data manipulation language** (**DML**) per consentire agli utenti di manipolare (cioè recuperare, inserire, cancellare e aggiornare) dati.
- Nei DBMS moderni si tende ad utilizzare un unico linguaggio integrato che comprende costrutti:
  - per la definizione di schemi concettuali;
  - per la definizione di viste;
  - per la manipolazione dei DB e
  - per la definizione della memorizzazione.
- ***Esempio:***
  - Il linguaggio di database relazionali **structured query language (SQL)** rappresenta una combinazione di DDL, VDL, DML e SDL.



I DML DI ALTO LIVELLO (O NONPROCEDURALE)

- Consentono da soli di specificare operazioni di database complesse in marniera concisa.

- In molti DBMS gli statement di DML di alto livello
  - Possono essere specificati interattivamente da terminale.
  - Possono essere inglobati (*embedded*) in un linguaggio di programmazione general-purpose (in questo caso gli statement DML sono identificati all'interno del programma in modo che il DBMS possa trattarli (*precompilazione*).

- I DML di alto livello (come SQL) sono detti **set-at-a-time** o **set-oriented** perché possono specificare e ritrovare molti record in un singolo statement DML (***es:*** l'SQL).
- Sono detti **dichiarativi** perché una query di un DML high-level spesso specifica **quale** dato deve essere ritrovato piuttosto che **come** ritrovarlo.

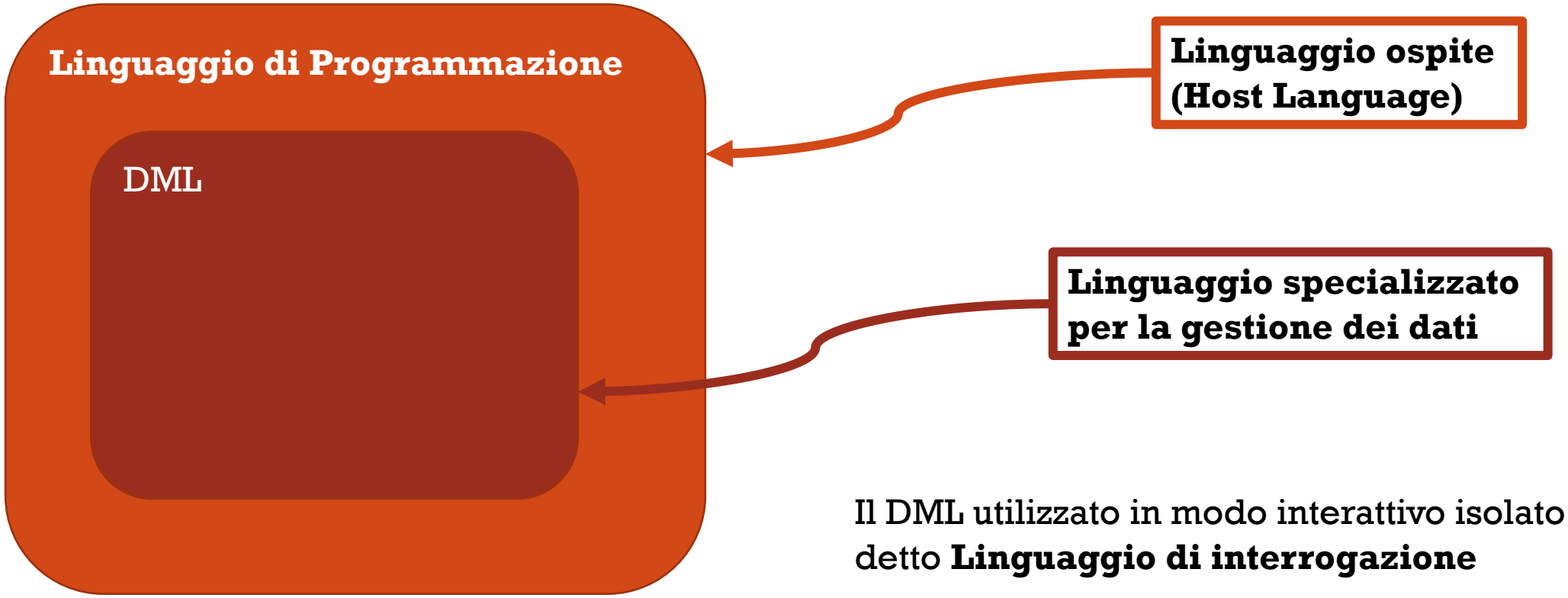


I DML DI BASSO LIVELLO (PROCEDURALE)

- **Deve** essere inglobato in un linguaggio general-purpose;
- È detto record-at-a-time perché tipicamente ritrova record individuali nei DB e tratta ciascun record separatamente;
- Ha quindi bisogno di usare costrutti di programmazione come l'iterazione per ritrovare e trattare ogni record da un insieme di record.



DML + LINGUAGGI DI PROGRAMMAZIONE



Il DML utilizzato in modo interattivo isolato è detto **Linguaggio di interrogazione**

SQL, UN LINGUAGGIO INTERATTIVO

- *"Trovare i corsi tenuti in aule a piano terra"*

Corsi

Corso	Docente	Aula
Basi di dati	Rossi	DS3
Sistemi	Neri	**N3**
Reti	Bruni	**N3**
Controlli	Bruni	G

Aule

Nome	Edificio	Piano
DS1	OMI	**Terra**
N3	OMI	**Terra**
G	Pincherle	Primo



SQL, UN LINGUAGGIO INTERATTIVO (2)

SELECT Corso, Aula, Piano

FROM Aule, Corsi

WHERE Nome = Aula **AND** Piano = *"Terra"*

Corso	Aula	Piano
Sistemi	N3	Terra
Reti	N3	Terra



SQL IMMERSO (EMBEDDED) IN LINGUAGGIO OSPITE

```
write('nome della citta"?'); readln(citta);
EXEC SQL DECLARE P CURSOR FOR
      SELECT NOME, REDDITO
      FROM PERSONE
      WHERE CITTA = :citta ;
EXEC SQL OPEN P ;
EXEC SQL FETCH P INTO :nome, :reddito ;
while SQLCODE = Ø do
 begin
    write('nome della persona:', nome, 'aumento?'); readln(aumento);
    EXEC SQL UPDATE PERSONE
                SET REDDITO = REDDITO + :aumento
                WHERE CURRENT OF P ;
    EXEC SQL FETCH P INTO :nome, :reddito ;
 end;
EXEC SQL CLOSE CURSOR P ;
```



INTERFACCE PER I DATABASE



INTERFACCE MENU-BASED

- Presentano all'utente liste di opzioni, dette menù, che guidano l'utente nella formulazione di una richiesta.

  La query è composta un passo alla volta scegliendo opzioni da una lista di menu che viene visualizzata dal sistema.
- Non c'è necessità di memorizzare i comandi specifici e la sintassi di un query language.
- I *menu pull-down* sono spesso usati nelle interfacce browsing, che consentono all'utente di guardare i contenuti del DB in maniera non strutturata.

INTERFACCE MOBILE

- Interfacce di accesso ai propri dati tramite mobile devices
- Le app che le implementano solitamente sono fornite di interfacce integrate che permettono una scelta limitata di opzioni rispetto alle interfacce Web.
- Permettono comunque operazioni sensibili quali pagamenti, prenotazioni, …



INTERFACCE FORM-BASED

- Un'interfaccia form-based visualizza un form per ciascun utente. Gli utenti possono:
  - Riempire tutte le entrate del form completamente per inserire nuovi dati.
  - Riempire solo alcune entrate, nel qual caso il DBMS ritroverà dati che fanno matching per il resto delle entrate.

- I form sono in genere progettati e programmati per utenti naive (non esperti).
- Il form specification language, che molti DBMS hanno, aiuta i programmatori a specificare i form.
  - ***Es:*** *SQL*Forms* è un Form-based Language che permette di definire interrogazioni per mezzo di form progettati insieme allo schema della base di dati
  - ***Es:*** *Oracle Forms* è un componente della suite Oracle per la progettazione e lo sviluppo di applicazioni tramite form



Ricerca avanzata

Trova pagine web che contengono...		Per fare questo nella casella di ricerca.
tutte queste parole:	database	Digita le parole importanti: `labrador retriever nero`
questa esatta parola o frase:		Racchiudi le parole esatte tra virgolette: `"labrador retriever"`
una qualunque di queste parole:		Digita `OR` tra tutte le parole che vuoi: `miniatura OR standard`
nessuna di queste parole:		Anteponi il segno - (meno) alle parole da escludere: `-roditore, - "Jack Russell"`
numeri da:	a	Inserisci due punti (..) tra i numeri e aggiungi un'unità di misura: `10..35 kg, € 300..€ 500, 2010..2011`

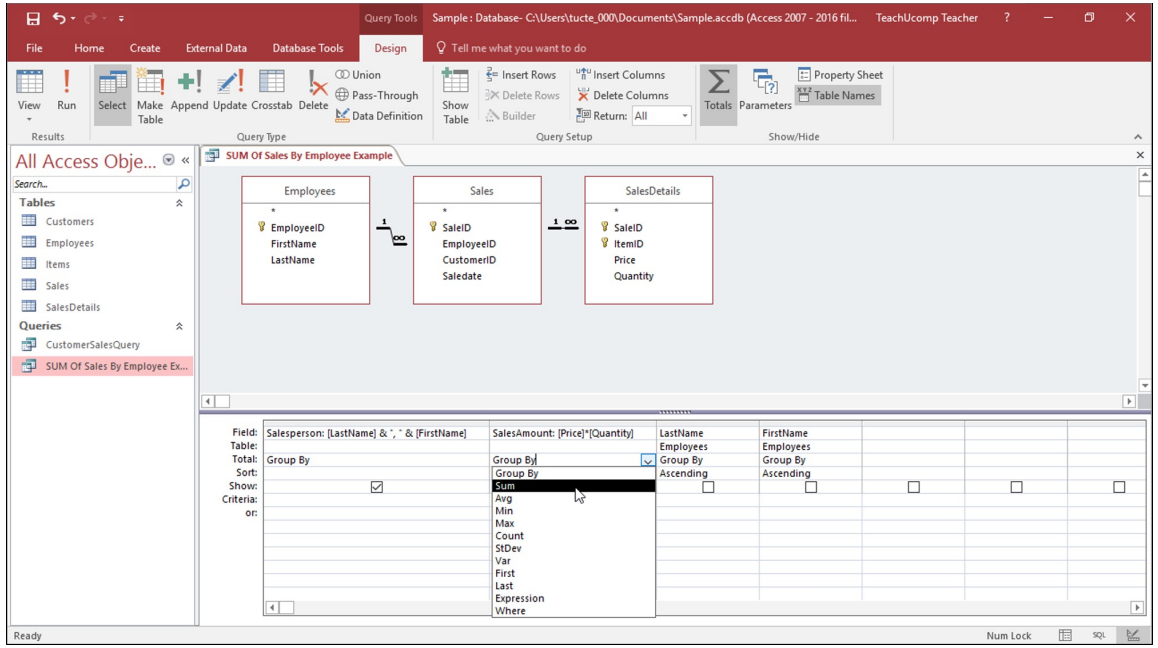
Poi limita i risultati per...

lingua:	tutte le lingue	Trova le pagine nella lingua selezionata.
area geografica:	tutti i Paesi	Trova le pagine pubblicate in un'area geografica specifica.
ultimo aggiornamento:	in qualsiasi data	Trova le pagine aggiornate nel periodo di tempo specificato.
sito o dominio:		Cerca in un sito (come `wikipedia.org`) o visualizza soltanto i risultati relativi a un dominio, come `.edu`, `.org` o `.gov`
termini che compaiono:	in un punto qualsiasi della pagina	Cerca i termini nell'intera pagina, nel titolo della pagina, nell'indirizzo web o nei link che rimandano alla pagina desiderata.
SafeSearch:	Mostra i risultati più pertinenti	Indica a SafeSearch se filtrare i contenuti sessualmente espliciti.
tipo di file:	qualsiasi formato	Trova le pagine nel formato che preferisci.
diritti di utilizzo:	risultati non filtrati in base alla licenza	Trova le pagine che puoi utilizzare liberamente.

Ricerca avanzata

INTERFACCE GRAFICHE (GUI)

- Tipicamente un'interfaccia grafica dispone su video uno schema in forma diagrammatica.
- L'utente specifica una query manipolando il diagramma, tramite dispositivi di puntamento, come il mouse.



INTERFACCE IN LINGUAGGIO NATURALE

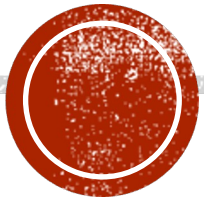
- Accettano richieste scritte in linguaggio naturale e cercano di *"capire".*
- Hanno un proprio schema specifico, che è simile allo schema concettuale del DB.
- Nell'interpretare le richieste l'interfaccia fa riferimento alle parole nello schema e a un insieme di parole standard (*dizionario*).
- se l'interpretazione ha successo, l'interfaccia genera una query high-level che corrisponde alla richiesta in linguaggio naturale e la sottomette al DBMS altrimenti inizia un dialogo con l'utente per chiarire la richiesta.
  - ***Es:*** Motori di ricerca come Google, AskJeeves, …
  - **Es:** SIRI



INTERFACCE BASATE SU KEYWORD

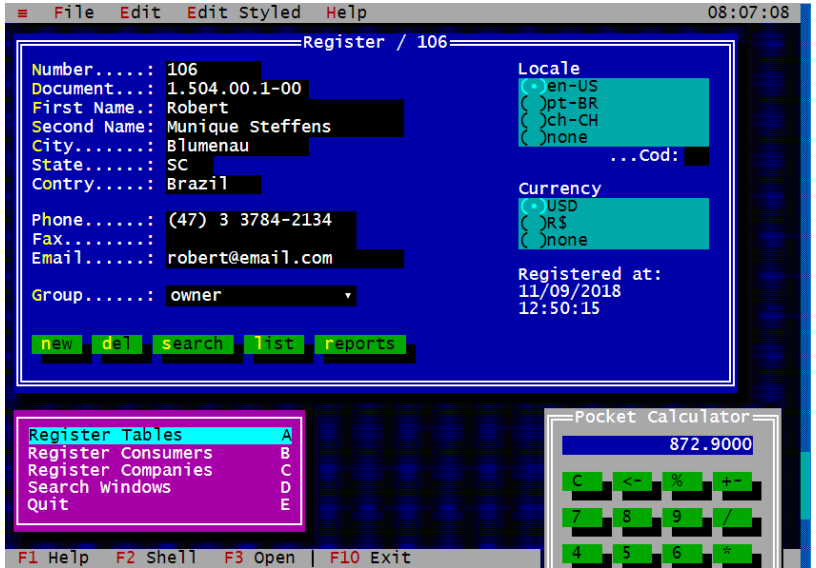


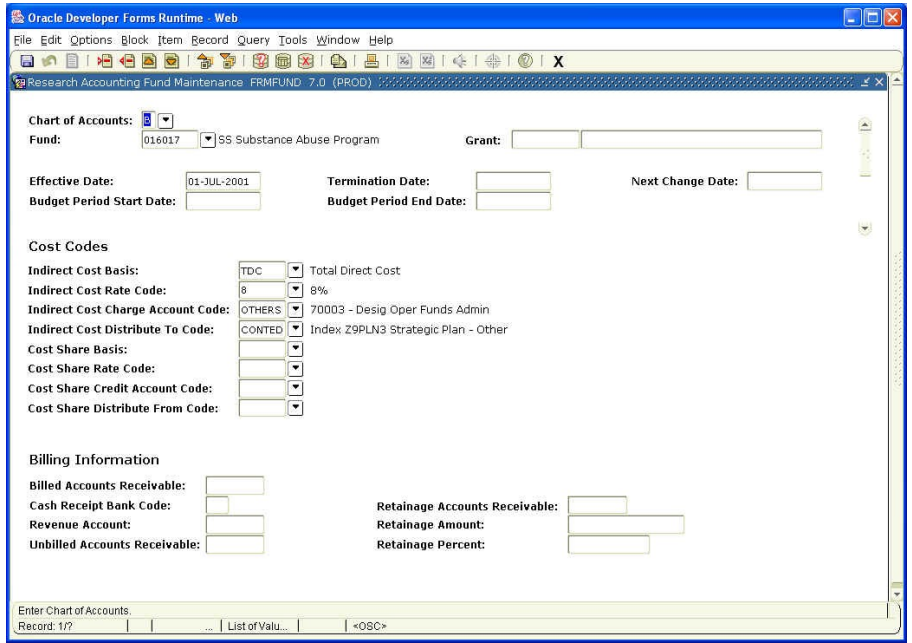


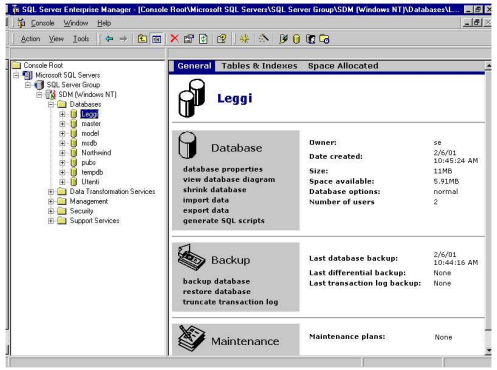


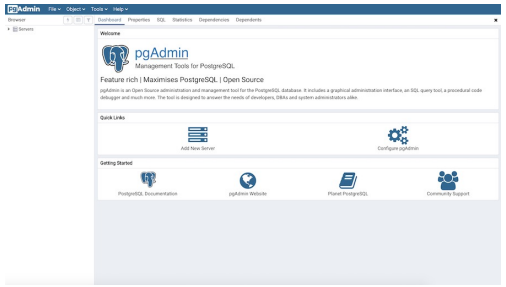
INTERFACCE PER UTENTI PARAMETRICI

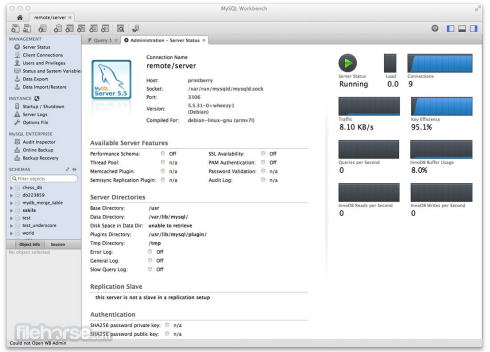
- Sono progettate per utenti naive (***es:*** impiegati di banca), così da comprendere un piccolo insieme di comandi abbreviati.
- Riguardano l'esecuzione ripetuta di un piccolo insieme di operazioni.
- Viene implementata una interfaccia speciale per ciascuna classe nota di utenti non esperti.

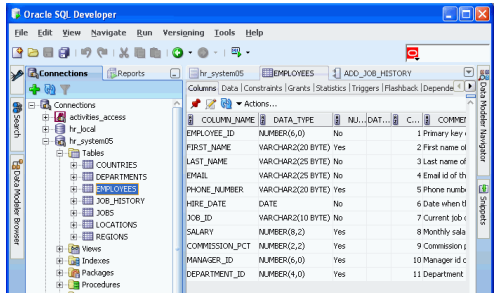












INTERFACCE PER I DBA

- La maggior parte dei DBMS contengono comandi privilegiati che possono essere usati solo dallo staff del DBA (***es:*** creare account, settare dei parametri, cambiare uno schema, riorganizzare la struttura di memorizzazione del DB).

ARCHITETTURE



ARCHITETTURE PER I DBMS

- Le architetture per i DBMS hanno seguito un evoluzione molto simile a quelle dei moderni sistemi di elaborazione

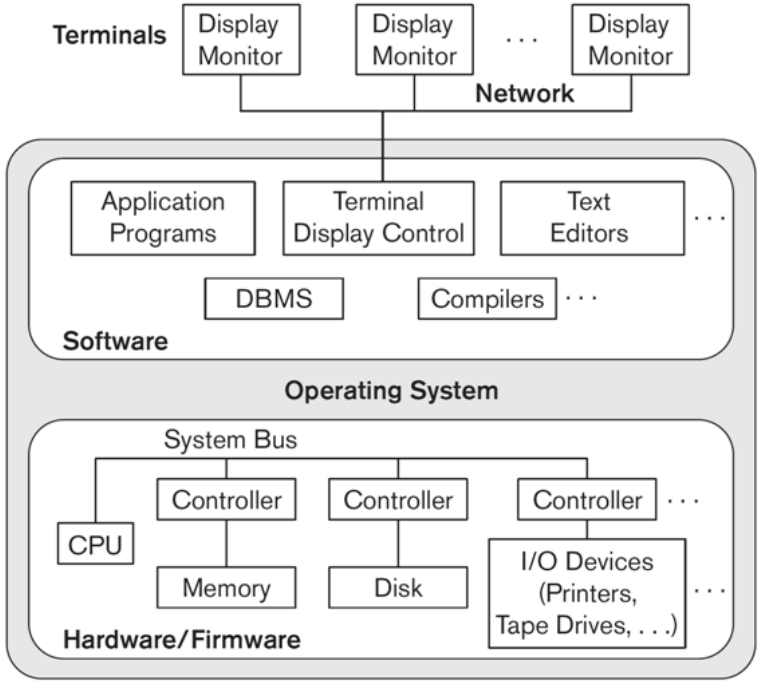
- In un primo momento si avevano sistemi mainframe, ai quali gli utenti accedevano con dei terminali «dumb», che fornivano solo operazioni di visualizzazione

- In un secondo momento, si è andato verso architetture modulari (client-server), dove una macchina funge da fornitore di servizi e le altre accedono e forniscono strumenti di visualizzazione (e alcune permettono anche l'elaborazione)
  - A due livelli
  - A tre livelli
  - A *n livelli*



ARCHITETTURA DEL DBMS CENTRALIZZATO

- Tutte le funzionalità del DBMS, l'esecuzione del programma applicativo, e l'elaborazione dell'interfaccia utente sono eseguite su una sola macchina.



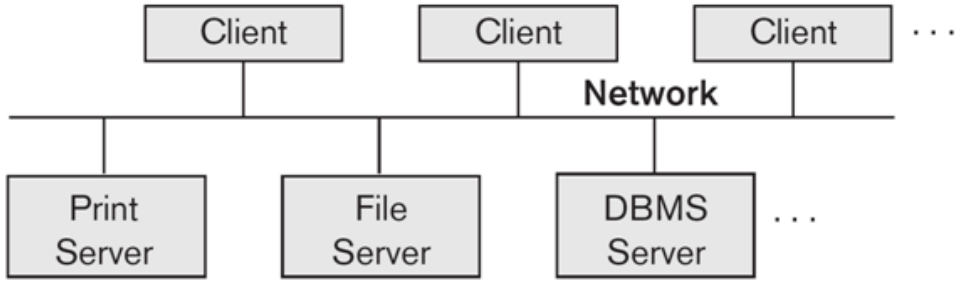
ARCHITETTURA CLIENT/SERVER BASE

- Sono **Server** con specifiche funzionalità:
  - **File server**
    - Mantiene e gestisce i file delle macchine client.
  - **Printer server**
    - È connesso a diverse stampanti,
    - Tutte le richieste di stampa dalle macchine client sono trasmesse a questa macchina.
  - **Web server** o **e-mail server**
- Le **macchine Client:**
  - Forniscono l'utente di un interfaccia appropriata per utilizzare i server.
  - Hanno capacità operazionali locali per eseguire applicazioni in locale.

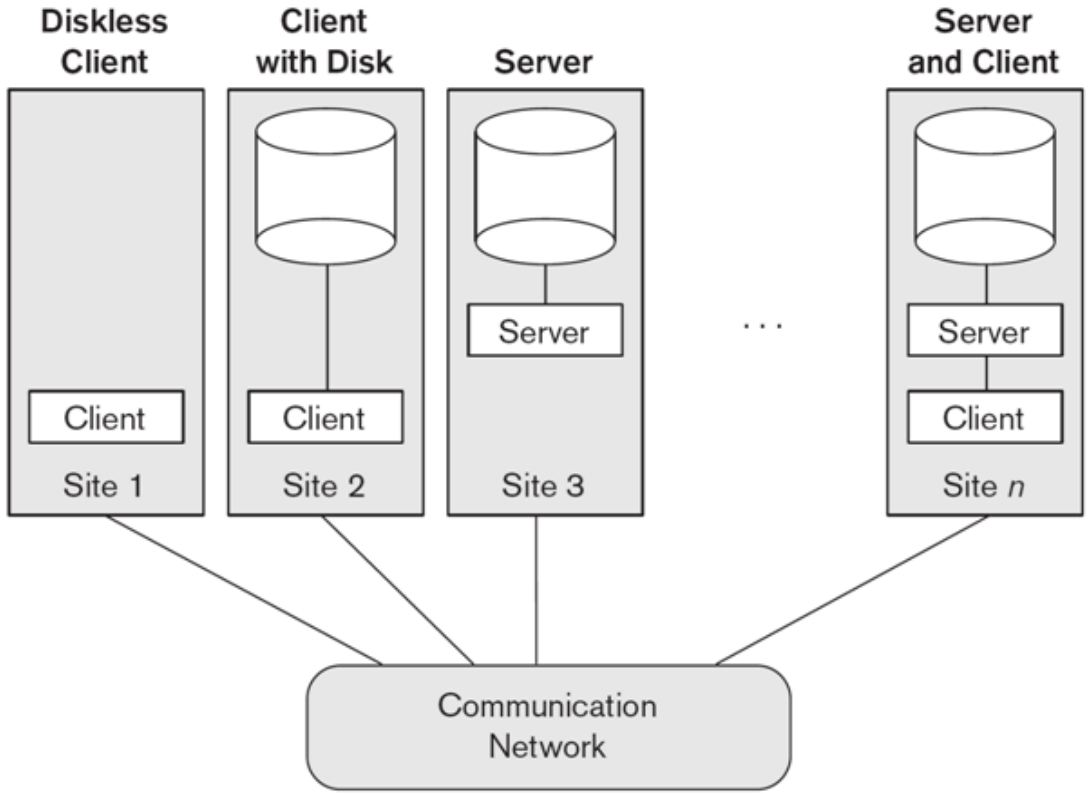


ARCHITETTURA CLIENT/SERVER TWO-TIER

- Architettura logica:



- Architettura fisica:



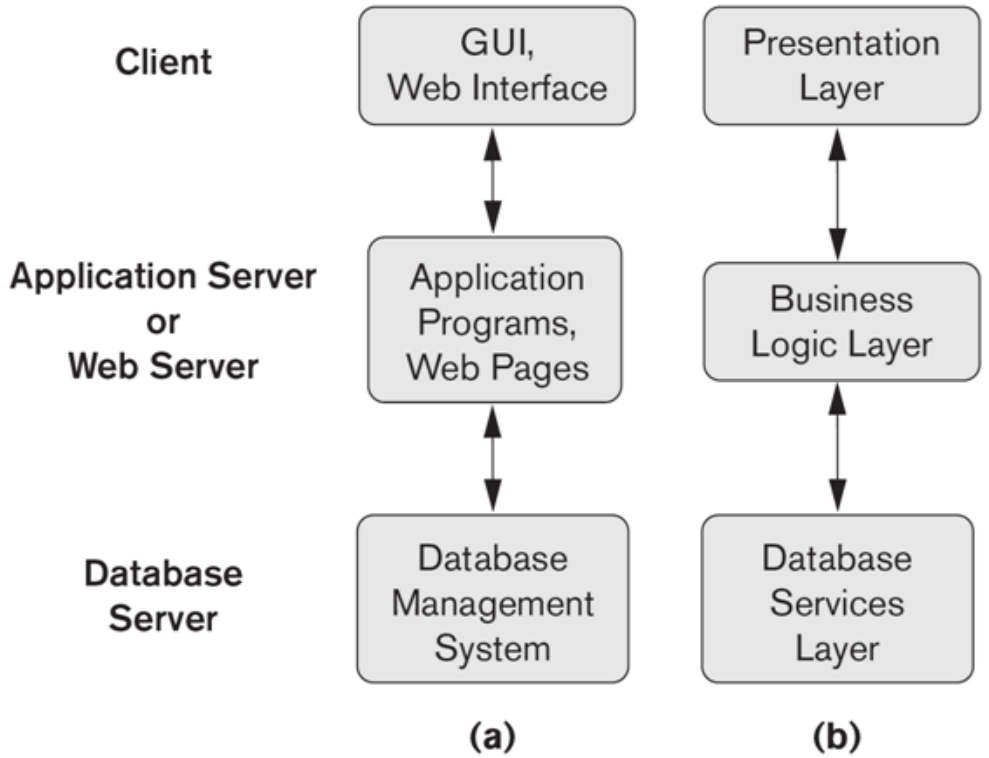
ARCHITETTURA CLIENT/SERVER TWO-TIER PER DBMS

- Applicativi e interfacce sono state le prime componenti ad essere spostati sul lato client.
  - Quando è richiesto l'accesso a un DBMS, viene stabilita una connessione
- Le funzionalità transazionali e di interrogazione rimasero lato server
  - Server delle interrogazioni/transazioni, Server SQL (nei R-DBMS)
- Open Database Connectivity (ODBC)
  - Fornisce una Application Programming Interface (API) che permette ad un programma in esecuzione sulla macchina client di comunicare con il DBMS
  - Entrambi le macchine client e server devono avere istallato il software necessario.
- Java DataBase Connectivity (JDBC)
  - Permette ai programmi client scritti in **Java** di accedere ad uno o più DBMS attraverso un interfaccia standard.



ARCHITETTURA CLIENT/SERVER THREE-TIER

- Application server o Web server
  - Aggiunge uno strato intermedio tra il client ed il database server.
  - Esegue i programmi applicativi e memorizza le regole di business.



INFO SULLA LEZIONE

- Tutto il capitolo 2 tranne il 2.4 e il 2.6
- Il 2.4 spiega l'ambiente di esecuzione dei DBMS
  - Quali sono i moduli che lo compongono
  - Quali tool e utility mettono a disposizione i DBMS attuali
- Il 2.6 effettua una classificazione dei DBMS
  - R-DBMS
  - R-DBMS a oggetti
  - Distributed DBMS (DDBMS)
  - Native XML DBMS (DBMS di tipo sperimentale)



FINE

Per eventuali domande: (in ordine di preferenza personale)

- Ora.
- Chat di Teams
- Mail: silvio.barra@unina.it